

Привет! Ошибка возникает из-за того, что Django пытается записать дату `datetime.datetime` в числовое поле (скорее всего, в `ForeignKey` связь с пользователем).

Это происходит из-за неверного значения по умолчанию (`default`), которое вы указали при создании/изменении поля в моделях.

Причина ошибки

В миграции `api.0002_course_user` вы добавляете или изменяете поле (вероятно, связь `user` в модели `Course`), и Django спросил у вас значение по умолчанию для старых записей. Вы случайно передали туда дату (например, `timezone.now` или `datetime.now`) вместо **ID пользователя** (числа).

Как исправить

1. Откройте файл миграции:

Найдите файл `api/migrations/0002_course_user.py` .

2. Найдите строку с ошибкой:

Ищите внутри операции `AddField` или `AlterField` аргумент `default=...` . Там будет что-то вроде `default=datetime.datetime(...)` или `default=django.utils.timezone.now` .

3. Исправьте значение:

1. Если это связь `ForeignKey` (например, на модель `User`), дефолтом должно быть число (ID существующего пользователя, например, `default=1`) либо сделайте поле nullable: `null=True, blank=True` (тогда `default` можно убрать).

4. Альтернативный быстрый путь (если база локальная и ее не жалко):

1. Удалите файл миграции `0002_course_user.py` .
2. Исправьте `default` в самом файле `models.py` для модели `Course` .
3. Заново запустите `python manage.py makemigrations` и затем `python manage.py migrate` .

Если вы покажете код модели `Course` из файла `models.py` или содержимое миграции `0002_course_user.py` , я смогу написать точную

строку для замены. Какое решение выберем?



"Как найти суперюзеров и их id в проекте?"

Найти всех суперпользователей и узнать их ID можно двумя быстрыми способами через терминал.

Перед выполнением убедитесь, что вы находитесь в папке проекта и у вас активировано виртуальное окружение `(.venv)`.

Способ 1: Через Django Shell (Самый простой)

Запустите интерактивную консоль Django:

```
bash
```

```
python manage.py shell
```

Используйте код с осторожностью.



Затем скопируйте и вставьте этот код (он выведет ID и юзернеймы всех суперпользователей):

```
python
```

```
from django.contrib.auth import get_user_model
User = get_user_model()
for user in User.objects.filter(is_superuser=True):
    print(f"ID: {user.id} | Username: {user.username}")
```

Используйте код с осторожностью.



Для выхода из консоли введите `exit()`.

Способ 2: Через команду создания (Если суперюзеров еще нет)

Если вы еще не создавали администратора, то база данных пока пуста. Вы можете сразу создать нового суперпользователя командой:

```
bash
```

```
python manage.py createsuperuser
```

Используйте код с осторожностью.



После создания его ID в базе данных гарантированно будет равен `1`.

Если в вашей базе уже есть администратор, какой **ID** у него оказался? Можем теперь вернуться к исправлению миграции.



Затем скопируйте и вставьте этот код (он выведет ID и юзернеймы всех суперпользователей):

"Сделал. Потом нужно enter несколько раз нажать?"

Да, всё верно.

В интерактивной консоли Django (Python shell) после вставки многострочного кода (например, цикла `for`) нужно нажать `Enter` **два** раза.

Почему это нужно?

1. **Первый Enter** перенесет вас на новую строку внутри цикла (консоль будет ждать продолжения ввода и покажет три точки `...`).
2. **Второй Enter** (на пустой строке) даст понять Python, что вы закончили писать блок кода, и запустит его на выполнение.

После этого на экране должен появиться список ваших суперпользователей. Получилось увидеть их **ID**?



"Получилось. Я тут еще бд заполняю. Можешь ошибки глянуть?..."



Отличный скрипт, задумка с `bulk_create` правильная! Но в коде есть три критические ошибки, из-за которых он упадет, и пара логических недочетов.

Вот разбор ошибок и готовый исправленный вариант.

Ошибки в коде:

1. Неверный аргумент в `lorem_ipsum` (Критическая)

Вы передали `common='False'` (строку в кавычках). Метод ожидает булево значение: `common=False` (без кавычек). Из-за этого Django выдаст ошибку.

2. Отсутствует обязательное поле `user` (Критическая)

В моделях `Course` поле `user` (автор курса) является обязательным. Для первого курса вы его указали (`user=user`), а для второго и третьего («Поколение Lorem» и «Ipsum») — забыли. Django выдаст `IntegrityError` (NOT NULL constraint failed).

3. Метод `lorem_ipsum.paragraphs` возвращает список (Логическая)

Этот метод возвращает список строк `['Параграф 1', 'Параграф 2']`. Если передать его напрямую в `description`, в базу запишется некрасивая строка с квадратными скобками: `['Lorem...', 'Ipsum...']`. Нужно объединить их через перенос строки: `'\n\n'.join(...)`.

4. Неправильное имя файла в комментарии (Заметка)

Команда вызывается как `python manage.py populate_db` (без расширения `.py` на конце).

Исправленный и улучшенный код:

Замените содержимое вашего файла на этот вариант:

```
python
```

```

from django.core.management.base import BaseCommand
from django.utils import lorem_ipsum
from api.models import User, Course, Step, StepElement, TextElement,
TestElement, TestOption, UserStepProgress

class Command(BaseCommand):
    help = 'Базовое заполнение таблиц тестовыми данными'

    def handle(self, *args, **kwargs):
        # 1. Получаем или создаем суперпользователя safely
        user = User.objects.filter(username='admin').first()
        if not user:
            user = User.objects.create_superuser(username='admin',
password='1234')
            self.stdout.write(self.style.SUCCESS('Создан
суперпользователь admin'))

            # Генерируем тексты для описания заранее, объединяя абзацы в
одну строку
            lorem_desc_1 = '\n\n'.join(lorem_ipsum.paragraphs(3,
common=False))
            lorem_desc_2 = '\n\n'.join(lorem_ipsum.paragraphs(3,
common=False))

            # 2. Готовим список курсов (теперь у всех указан user)
            courses = [
                Course(
                    title='Поколение Python',
                    description='В курсе рассказывается об основных типах
данных, конструкциях и принципах структурного программирования языка
Python.',
                    user=user,
                ),
                Course(
                    title='Поколение Lorem',
                    description=lorem_desc_1,
                    user=user, # Передаем пользователя
                ),
                Course(
                    title='Поколение Ipsum',
                    description=lorem_desc_2,
                    user=user, # Передаем пользователя
                ),
            ]

            # 3. Сохраняем в базу одной пачкой
            Course.objects.bulk_create(courses)

```

```
# Информировать в консоль, что всё прошло успешно
self.stdout.write(self.style.SUCCESS(f'Успешно создано
курсов: {len(courses)}'))
```

Используйте код с осторожностью.



Что делать дальше?

1. Убедитесь, что файл называется `populate_db.py` и лежит строго в папке `ваше_приложение/management/commands/`.
2. Запустите его командой:

```
bash
```

```
python manage.py populate_db
```

Используйте код с осторожностью.



Вы планируете дальше наполнять эти курсы **шагами (Step)** и **элементами (TextElement, TestElement)** внутри этого же скрипта? Если нужно, я могу набросать пример, как связать их через `bulk_create`, учитывая, что у созданных через `bulk_create` объектов изначально нет ID в коде.



"сделал. После этого надо делать makemigrations?"

Нет, делать `makemigrations` и `migrate` сейчас **не нужно**.

Почему?

- `makemigrations` нужен только тогда, когда вы меняете структуру таблиц (код в файле `models.py`) — например, добавляете новое поле, удаляете модель или меняете тип данных.

- Ваш скрипт `populate_db` не меняет структуру. Он просто берет готовую структуру и **заполняет её данными** (как если бы вы зашли в админку и руками создали курсы).

Вы можете запускать команду `python manage.py populate_db` в любой момент, когда хотите добавить тестовые данные в базу.

Скрипт выполнен без ошибок? Хотите дополнить его **созданием шагов (Step)** для этих курсов?



"Все сработало, но я сделал ошибку. Как мне удалить или изменить descriptions у одной из строк таблицы?"

Самый быстрый способ исправить описание одной строки — сделать это через интерактивную консоль Django (`shell`).

Вам не нужно переписывать базу заново, достаточно обновить конкретную запись по её названию.

Шаг 1: Откройте консоль Django

Выполните в терминале команду:

```
bash
```

```
python manage.py shell
```

Используйте код с осторожностью.



Шаг 2: Выполните код для обновления

Скопируйте и вставьте этот код в консоль (замените текст в кавычках на нужный вам) и нажмите **Enter два раза**:

```
python
```

```
from api.models import Course

# Находим курс по названию и обновляем его описание
Course.objects.filter(title='Поколение Python').update(
    description='Ваш новый правильный текст описания курса.'
)
```

Используйте код с осторожностью.



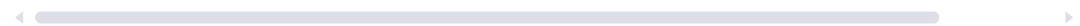
Что делать, если нужно полностью очистить базу и заполнить заново?

Если вы хотите исправить текст прямо в файле скрипта `populate_db.py` и перезапустить его, то сначала нужно удалить старые записи, чтобы они не дублировались.

Для этого добавьте в самое начало метода `handle` (прямо перед получением пользователя) строку очистки:

python

```
Course.objects.all().delete() # Удалит все старые курсы перед создани
```



Используйте код с осторожностью.



Вы хотите просто **отредактировать** этот один курс или планируете **добавить очистку** базы в сам скрипт, чтобы запускать его «с чистого листа»?



"Спасибо! Оч помог."

Рад был помочь! Отличная работа, с заполнением базы через кастомные команды всегда удобнее тестировать проект.

Если в процессе связывания курсов с шагами (`Step`) или элементами возникнут трудности с получением ID — пишите, разберемся.

Удачи с дипломным проектом! На каком этапе планируете **продолжить разработку** (добавление шагов, тестов или настройка API)?

